

Language Models Need Sleep — 한국어의역본

이 문서는 arXiv:2606.03979 「Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories」 (Behrouz, Hashemi, Mirrokni (Google / Cornell)) 의 한국어의역본입니다. 스터디용이며 공식 번역이 아닙니다. 수식 · 수치 · 그림은 원문 기준이고, 그림은 원저자의 것입니다. 자연스러운 이해를 위해 문장은 의역했습니다.

초록 (Abstract)

지난 수십 년간 기계학습 알고리즘 설계는 크게 발전해 왔다. 초창기의 과제특화형 얇은 모델에서 시작해, 오늘날에는 훨씬 범용적인 대규모 언어모델 (LLM) 에 이르렀다. 그러나 이런 모델들은 즉각적인 예측이나 in-context learning 이 필요한 과제에서는 인상적인 성능을 보이면서도, **지속적으로 (continually) 학습하고 문맥에서 얻은 일시적 지식을 장기 파라미터로 효과적으로 옮기는 능력**은 여전히 갖추지 못하고 있다.

우리는 인간의 학습 과정에서 영감을 받아, 모델이 지속적으로 학습하고, 불안정한 단기 기억을 replay 와 함께 안정적인 장기 지식으로 증류하며, “Dreaming(꿈꾸기)” 과정을 통해 스스로를 재귀적으로 개선할 수 있게 해주는 “**Sleep(수면)**” 패러다임을 제안한다. 좀 더 구체적으로, 수면은 두 단계로 구성된다.

1. **기억 공고화 (Memory Consolidation): Knowledge Seeding(지식 파종)** 이라 부르는 상향식 증류 과정으로, 더 작은 자기 자신의 기억을 더 큰 네트워크로 증류하여 지식을 보존하면서도 용량을 늘린다. 개념 증명 (proof of concept) 으로, 우리는 Knowledge Seeding 을 위한 새로운 일반화 증류 과정 (on-policy distillation 과 강화학습 (RL) 기반 imitation learning 의 결합) 을 제시한다.
2. **꿈꾸기 (Dreaming)**: 자기 개선 단계로, 모델이 RL 을 사용해 합성데이터로 이루어진 커리큘럼을 스스로 생성하여 새 지식을 되새기고 (rehearse) 기존 역량을 인간의 감독 없이 다듬는다.

long-horizon, continual learning, 지식 통합 (knowledge incorporation), few-shot 일반화 과제에 대한 실험은 수면 단계의 중요성을 뒷받침한다.

1 서론 (Introduction)

대규모 언어 모델 (LLM) 의 등장은 기계학습 연구의 결정적 이정표다. 즉, 과제특화 모델에서 다양한 창발적 (emergent) 능력을 지닌 더 범용적인 시스템으로의 패러다임 전환이다. LLM 은 다양한 과제에서 놀라운 능력을 보이지만, **최초 배포 이후에는 대체로 정적 (static)** 이다. 다시 말해 사전 학습 · 사후 학습 때 익힌 과제는 잘 수행하지만, 자신의 즉각적인 문맥을 넘어선 새로운 능력을 지속적으로 습득하지는 못한다. 이본질적인 정적 성질은 치명적인 취약점을 낳는다. 모델의 지식과 기술은 점점 낡아 가고, 고정된 “지식 컷오프 (knowledge cutoff)” 시점 이후의 새로운 사실 · 사건 · 변화하는 정보에 대해서는 무지한 채로 작동한다.

이 한계를 극복하려는 노력은 주로 두 갈래였다. (1) 확장된 데이터셋으로 **재사전학습 (re-pretraining)** 하는 방법인데, 효과적이긴 하나 계산비용이 커서 잦은 업데이트에는 비현실적이다. (2) 값비싼 지속적 파라미터 업데이트나, 파인튜닝 · 저랭크 적응 (LoRA) 같은 경량 대안인데, 이들은 반복 업데이트 시 흔히 **파국적 망각 (Catastrophic Forgetting, CF)** 을 일으킨다. CF 란 모델이 새 과제를 학습하면서 기존 과제에 대한 숙련도가 급격히 무너지는, 잘 알려진 현상이다. 지식 노후화가 한쪽 뿐이고, 파국적 망각과 업데이트의 과도한 비용 · 파괴적 성질이 다른 쪽 뿐인 이 딜레마는 아직 풀리지 않은 핵심 과제를 부각한다. 바로 **LLM 이 자신의 생애 주기 전반에 걸쳐 점진적이고 효율적으로 학습하도록 하는 것**이다.

최근에는 **In-Context Learning(ICL)** 이 매우 효율적이면서도 성공적인 형태의 지속 학습으로 주목받아 왔다. 처음 ICL 은 대규모 데이터로 학습된 LLM 의 창발 능력으로 알려졌다. 즉 문맥에 빠르게 적응하여 zero-/few-shot 과제를 수행하는 능력이다. 이후 연구들은 ICL 을 일종의 **메타학습 (meta-learning) 과정**으로 형식화했다. 모델이 시퀀스를 따라 내부 연산을 수행하며 문맥 지식을 단기 기억에 유지하거나 압축해 넣음으로써 자신의 출력에 반영한다는 것이다. ICL 은 지속 학습으로서 효과적·효율적이지만, **시퀀스 모델의 문맥창 (context window) 에 국한된다**. 즉 새로 습득한 지식은 세션/문맥이 끝나는 순간 모델에서 사라진다. 이는 하나의 결정적 질문을 던진다. **모델은 어떻게 취약한 단기 기억을 더 안정적인 장기 지식으로 효과적으로 옮길 수 있는가?**

비유로, Behrouz et al. (2025) 이 LLM 에 적용한 **순행성 기억 상실 (anterograde amnesia)**의 예를 든다. 인간에게서 단기 기억을 장기 기억으로 옮기는 과정이 손상되는 경우가 있는데, 그 예가 순행성 기억 상실이다. 이는 질환 발병 이후에는 새 기억을 형성하지 못하지만 기존 기억은 온전히 남는 신경학적 상태다. 이런 조건에서는 사람의 지식이 단기 기억에 담기는 즉각적 현재와, 발병 이전의 먼 과거로 제한되며, 그 결과 매 순간의 현재를 마치 언제나 처음인 것처럼 반복적으로 경험하게 된다. 현재의 Transformer 기반 LLM 의 기억 처리에서도 유사한 패턴이 보인다. LLM 의 지식은 (1) 문맥 창에 담기는 즉각적 문맥 (즉 ICL), 또는 (2) “사전 학습의 종료” 이전의 먼 과거를 저장하는 MLP·투영 (projection) 층 둘 중 하나에 국한된다. 이 유사성은 다음 질문으로 이어진다. **인간 학습 과정에서 기억을 공고화하는 핵심 요소는 무엇인가?**

인간의 뇌는 기억을 공고화할 때 매우 효율적이며, 손실적 (lossy) 이지만 효과적이다. 이는 흔히 **신경가소성 (neuroplasticity)**—새로운 경험·기억·학습에 반응해 뇌 스스로를 바꾸는 능력—에 기인한다고 여겨진다. 최근 연구들은 장기 기억 형성이 서로 구별되지만 상호 보완적인 최소 두 가지 공고화 과정을 포함한다고 뒷받침한다. (1) 학습 직후, 심지어 깨어 있는 동안에도 일어나는 빠른 **“온라인 (online)” 공고화**로, 새롭게 취약한 기억 흔적이 안정화되어 단기에서 장기 저장으로 옮겨지기 시작한다. (2) 최근에 부호화된 패턴의 replay 를 반복하고 기억을 재조직하여 대뇌 피질 부위의 전이를 돕는, 수면 중에 일어나는 **“오프라인 (offline)” 공고화** (시스템 공고화라고도 함) 이다.

인간의 온라인 공고화 (활동/깨어 있는 단계) 인간의 온라인 공고화는 깨어 있는 상태에서 회상 (recall) 중에 기억을 재활성화함으로써 새로 학습한 기억을 능동적으로 강화·변형하는 것으로, 시간이 지날수록 기억을 더 안정적이고 더 의미 (semantic) 에 가깝게 만든다.

인간의 오프라인 공고화 (수면 및 조용한 휴식 단계) 인간의 오프라인 공고화는 학습 이후 휴식이나 수면 중에 일어나며, 새로 부호화된 기억이 안정화되고 복잡한 손실적 증류 과정을 통해 점차 더 분산된 신경질표상으로 변형된다.

순행성 기억 상실의 비유로 돌아가면, 증거는 이 조건이 두 단계 모두에 영향을 줄 수 있음을 시사한다. Behrouz et al. (2025) 은 최근 **Nested Learning(NL)** 패러다임을 제시하며 첫 번째 형태의 기억 공고화, 즉 온라인 공고화를 겨냥했다. 특히 NL 기반 **Hope 아키텍처** (그림 1 참고) 는 연속체 기억 시스템 (continuum memory system) 에서 기억을 재활성화하는데, 각 구성요소가 서로 다른 빈도로 그러나 end-to-end 로 업데이트되면서 빠르고 불안정한 구성요소의 지식을 더 안정적인 저빈도 모듈로 온라인으로 전이한다. 이 end-to-end 과정은 지식을 더 안정적인 구성요소로 직접 옮겨 망각을 늦출 수 있지만, 여전히 **동일한 양의 모델 용량을 소모**한다. 이는 모델이 추가적인 손실 압축 없이 지식을 동일한 추상화 수준으로 유지하기 때문이다. 나아가 이 형태의 공고화만으로는 강건한 지속 학습에 충분치 않다. (i) **선택적이며 회상 의존적 (retrieval-dependent)** 이다. 능동적 회상에 의존하므로 반복적으로 상기되는 기억 부분만 강화한다. (ii) **문맥 의존적 (context-dependent)** 이다. 온라인 공고화가 일으키는 업데이트는 오직 모델의 문맥에 기반하므로, 새 지식과 기존 지식의 더 높은 수준의 통합을 놓친다.

본 논문에서는 두 번째 형태의 공고화, 즉 인간의 수면 단계에서 영감을 받은 **수면을 통한 오프라인 공고화**에 초점을 둔다.

인간 학습 과정에서 수면의 역할 수면은 수동적 상태가 아니라 인지 기능에 필수적인, 역동적이고 고도로 구조화된 뇌 활동 시기다. 수면 중 뇌는 학습, 신경가소성, 자기 개선, 기억 공고화에 근본적인 복잡한 과정을 조율한다. 인간에서 이 과정은 주로 교대로 나타

나는 두 가지 결정적 수면단계, 즉 REM(급속 안구 운동) 수면과 Non-REM(NREM) 수면에 의해 지배된다.

비급속 안구 운동 수면 (서파 수면, Slow-Wave Sleep). 이 단계, 특히 서파 수면이라 불리는 가장 깊은 국면은 동기화된 고진폭 · 저주파 신경 활동이 특징이다. 서파 수면은 학습에 결정적인 두 가지 주요 기능과 연관된다. 첫째는 **시냅스항상성 (synaptic homeostasis)** 으로, 깨어 있는 경험에서 순증한 연결성을 상쇄하기 위해 시냅스 강도를 전역적으로 하향 조정 (downscale) 하여 대사균형을 유지하고 신경 포화를 방지한다. 둘째 핵심 기능은 **기억공고화**로, 취약하고 최근의 경험을 안정적인 장기 지식으로 변형한다. 이 과정은 **해마 (hippocampus) 와 신피질 (neocortex) 사이의 정교한 대화**로 조율된다. 해마는 특정한 일상 경험을 빠르게 부호화하는 고충실도의 임시 저장 시스템이고, 신피질은 그 경험에서 일반화된 규칙과 의미 지식을 점진적으로 학습하는 데 더 적합한 방대한 장기 저장소다. 서파 수면 중 뇌는 이 두 구조 사이에서 밤마다의 대화를 시작하여 정교한 정보 전이를 촉진한다. 주목할 점은, 이 전이가 단순히 원 (raw) 데이터를 replay 하는 것이 아니라, 깨어 있는 동안 얻은 지식을 **재구성 (re-architect)** 하여 추상을 추출하고 이를 하나의 응집된 의미 네트워크로 통합한다는 것이다.

급속 안구 운동 수면 (REM Sleep). 깨어 있는 상태와 닮은 고주파 · 저진폭 뇌파가 특징인 REM 수면은 가장 흔히 꿈꾸기와 연관된다. 기능적으로 이 단계는 **새로 형성된 시냅스의 선택적 강화**, 그리고 새 정보를 기존의 정서 · 의미 네트워크와 통합하는 것과 연관된다. 나아가 적응적 행동을 개선하기 위해 **미래시나리오를 시뮬레이션**하는 역할을 한다고 가설이 세워져 있다.

요약하면, 밤 동안 NREM 과 REM 단계가 주기적으로 교대하는 것이 결정적이다. NREM 수면은 하루의 경험을 공고화 · 가지치기 (prune) 하여 더 효율적인 지식 기반을 구축하는 것으로 보인다. 이어서 REM 수면은 이 정제된 기반 위에서 작동하여 새로운 연결을 탐색하고 두드러진 신경 경로를 강화하는 것으로 보인다.

지속 학습에는 학습 시간도 시험 시간도 없다 지속 학습자에게는 학습 시간과 시험 시간 사이에 뚜렷한 경계나 구분이 없다. 모델은 오직 두 가지 다른 상태만 경험한다. 입력으로 정보를 받을 때, 또는 고립된 학습 시스템일 때다.

지속 학습의 활동 (깨어 있음) 과 수면 단계 활동/깨어있는 시간에 지속 학습자는 새 입력 데이터를 받아 필요에 따라 처리한다. 그러나 수면 시간에는 입력 데이터를 최소한으로 (또는 전혀) 받지 않으며, 기억을 공고화하고 스스로를 개선하기 위해 기존 지식을 처리하는 데 집중한다.

기여 (Contributions)

인간의 기억 처리에서 영감을 받아, 우리는 지속 학습자에게는 학습 시간도 시험 시간도 없다고 주장한다. 대신 모델은 “활동” 상태와 “수면” 상태라는 두 단계를 주기적으로 가져야 한다. 활동 상태에서 모델은 새 데이터를 받아 처리하고, 수면 단계에서는 내부 지식과 최근 기억의 공고화에 집중한다. 이를 위해 우리는 LLM 을 위한 **수면 패러다임의 한 사례**를 제시하며, 이는 통합된 두 단계로 구성된다.

- **i. 기억 공고화 (Memory Consolidation):** 파국적 망각 (CF) 을 완화하고 더 높은 수준의 추상을 포착하기 위해, 기억 공고화 단계에서 모델은 새 파라미터를 활성화/잠금 해제하고 고빈도 (빠르게 업데이트되는) 층/모듈의 지식을 더 안정적인 저빈도 층/모듈에 새로 잠금 해제된 파라미터로 증류하는 주기적 과정을 사용한다. 이 과정은 새 파라미터에는 충분한 가소성을 허용하면서 기존 파라미터의 안정성을 보장하여 옛 지식을 보존한다.
- **ii. 꿈꾸기를 통한 자기 개선 (Self-Improvement via Dreaming):** 첫 단계가 지식 추상을 장기 기억으로 옮기는 것을 보장하는 반면, 이 단계는 재귀적 자기 개선 과정을 담당한다. 구체적으로, 모델은 현재 상태를 바탕으로 일련의 꿈 (dream, 즉 합성적으로 스스로 생성한 데이터) 을 생성하여, 특히 최근에 추가된 지식에 대한 숙련도를 높이는 데 초점을 두고 자신의 성능을 개선한다.

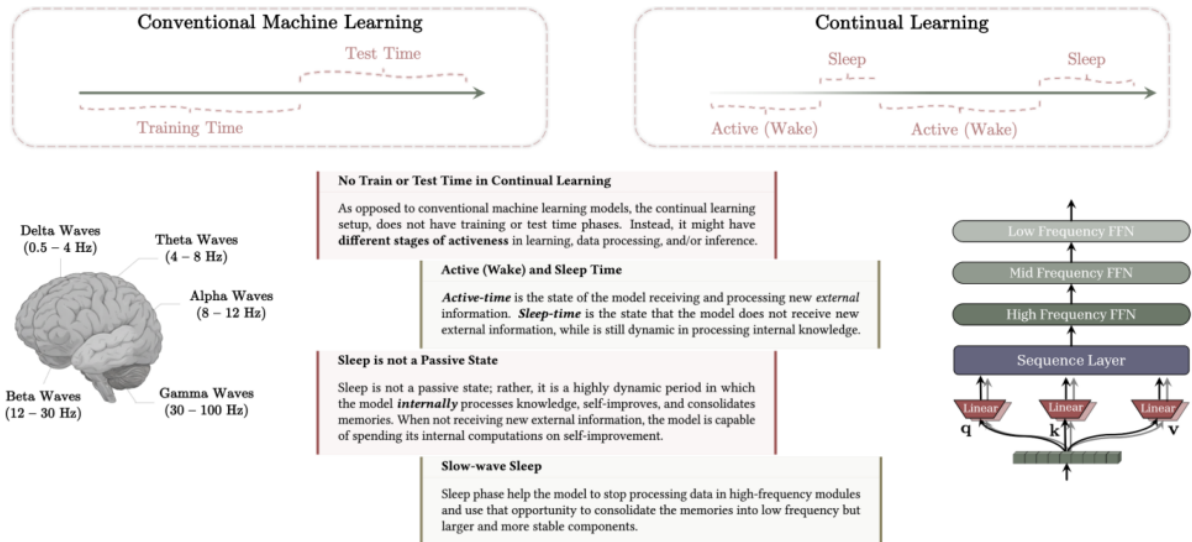


그림 1: 전통적 기계학습 대 지속 학습. 전통적 기계학습에서는 모델의 생애가 학습 시간과 시험 시간으로 나뉘지만, 지속 학습에는 이런 구분이 없다. 우리는 지속 학습자가 학습에서 서로 다른 활동 단계, 즉 (i) 활동/깨어 있는 시간과 (ii) 수면 시간을 가져야 한다고 제안한다. 수면 시간은 수동적 상태가 아니라, 데이터를 내부적으로 처리하여 빠르고 불안정한 모듈의 기억을 더 안정적인 저빈도 (느린) 구성요소로 공고화하는 시간이다. 출처: 원저자, arXiv:2606.03979 Fig.1

기술적 관점에서, 위 두 단계 각각에 대한 우리의 기여는 다음과 같다.

1. **주기적 파라미터 (비) 활성화 (Periodic Parameter (De)Activation):** 각 구성요소가 자체 업데이트 빈도를 갖게 하는 Nested Learning(NL) 패러다임 위에서, 우리는 주기적 · 점진적 파라미터 (비) 활성화 과정을 제안한다. 어떤 블록에 대해, 각 수면스텝마다 더 빠른 (고빈도) 블록의 파라미터 집합을 비활성화하고 이를 현재블록의 새로 활성화된 파라미터 집합으로 대체한다. 이로써 가소성을 유지하면서 기존 파라미터와의 지식 간섭을 피한다.
2. **Knowledge Seeding(상향식 종류):** 하나 또는 몇 개의 더 작은 모델이 더 큰 모델로 지식을 증류하는 새로운 지식 전이 방식인 **knowledge seeding** 을 제시한다. 이 설계는 큰 모델이 작은 모델들의 기존 지식을 보존하면서도 자신의 더 큰 용량을 활용하게 한다. KS 형식화를 바탕으로, 더 작은 버전의 모델 (예: 일부 파라미터가 비활성) 이 더 큰 버전의 모델 (파라미터가 활성화) 로 지식을 증류하는 **self-Knowledge Seeding(SKS)** 을 제시한다. 그런 다음 SKS 를 기억공고화의 해법으로 사용하여, 모델이 고빈도 층/블록에서 저빈도 블록으로 지식을 증류하게 한다.
3. **Imitation Learning 을 결합한 Generalized Knowledge Distillation(GKD):** SKS 형식화는 일반적이어서 어떤 목적 함수 · 지식 전이 방식도 쓸 수 있다. 여기서는 on-policy distillation 과 imitation learning 과정을 결합한 새 목적 함수를 제시한다. 우리의 imitation learning 에서는 교사 (즉 (압축된) 특권 정보를 가진 더 작은 버전의 모델) 가 합성 데이터를 생성한 뒤 시퀀스를 마스킹하고, 학생이 그 시퀀스의 이어질 부분을 예측하며, 교사가 생성한 데이터와 자신의 예측 사이의 거리에 기반해 보상을 받는다.
4. **실험적 평가:** 수면 패러다임의 효과를 일련의 도전적 하위 과제, 즉 (1) 사실 지식 통합, (2) few-shot 학습, (3) long-context 이해, (4) 지속 학습에서 평가한다. 결과는 수면 패러다임의 효과와, 지속 학습에서 반복적 지식 증류로 파라미터를 늘리는 것의 중요성을 뒷받침한다.

2 예비 지식 및 문제 정식화 (Preliminaries and Problem Formulation)

2.1 표기 (Notation)

굵은 소문자 (각각 대문자) 는 벡터 (각각 행렬) 를 나타내고, 아래첨자 t 는 시각 t 에 대응하는 상태를 가리킨다. 모듈 파라미터의 위첨자 (각각하이퍼파라미터의 위첨자) 는 모듈의 업데이트 빈도 (각각 서로 다른 인스턴스의 구별) 를 나타낸다. 본문 전반에서 $x \in \mathbb{R}^{L \times d_{in}}$ 을 입력, K, V, Q 를 시퀀스모델의 키·값·쿼리 행렬, L 을 시퀀스 길이로 둔다. 필요할 때 언어 모델 LM_{θ} 를 다음과같이 파라미터화한다.

$$\theta = \{W_1^{(f_1)}, \dots, W_{k_1}^{(f_1)}\} \cup \{W_1^{(f_2)}, \dots, W_{k_2}^{(f_2)}\} \cup \dots \cup \{W_1^{(f_c)}, \dots, W_{k_c}^{(f_c)}\}$$

여기서 파라미터 집합들은 가중치 업데이트 빈도 $f_1 \geq \dots \geq f_c$ 에 따라 정렬된다 (정의 1 참고). 이 식은 모델전체 파라미터를 업데이트 빈도별 그룹으로 나눠 정렬한 것을 뜻한다.

2.2 연속체 기억 시스템 (Continuum Memory System)

Transformer 아키텍처는 두 가지 핵심 구성요소로 이루어진다. (1) 연상기억 (associative memory) 으로 작동하여 문맥의 과거 토큰에 출력을조건화하고 그 결과 ICL 능력을 낳는 **어텐션 모듈**, 그리고 (2) 학습이후 고정되어 사전학습에서 얻은 지식을 부호화하는 **MLP(피드포워드)** 층이다. Behrouz et al. (2025) 이 논한 대로, 이런아키텍처는 **2 단계 기억 시스템**으로 해석할 수 있다. 어텐션의업데이트 범위는 문맥 길이이고 (즉 문맥이 끝나면 대응 파라미터가업데이트되고 습득한 지식은 망각된다), MLP의 업데이트 범위는 존재하지않는다 (즉 사전학습 이후 업데이트 없음). 이 관점에서 두 구성요소는 빈도스펙트럼의 양극단이며, 어텐션은 **무한 (infinite)** 업데이트 빈도, MLP는 **영 (zero)** 업데이트 빈도를 갖는다.

이 직관 위에서 Behrouz et al. (2025) 은 **연속체 기억시스템 (Continuum Memory System, CMS)** 을 제시했다. 아키텍처는 어텐션 같은시퀀스 모델 뒤에 각자 자체 빈도로 업데이트되는 MLP 층들의 사슬이 이어진형태다. 가장 느린 모듈의 한 번 업데이트에 걸리는 시간을 시간의 단위로삼고, 다른 구성요소의 업데이트율을 이에 상대적으로 정의한다.

정의 1 (업데이트 빈도). 임의의 가중치 구성요소 W 에 대해, 그빈도 f_W 를 단위 시간당 업데이트 횟수로 정의한다.

이 개념을 더 잘 이해하기 위해 입력이 길이 L 의 시퀀스인 **Fast-weight Programs(Schmidhuber 1992)** 의 간단한 예를 든다. 이경우 느린 가중치 (slow-weight) 의 매 스텝 (시간 단위) 마다 빠른가중치 (fast-weight) 는 L 번 업데이트되므로, 업데이트 빈도는 L 이된다.

이 빈도 정의를 따라, CMS 는 MLP 블록들의 사슬 $MLP^{(f_1)}(\cdot), \dots, MLP^{(f_k)}(\cdot)$ 로형식화되며, 각 블록은 청크 크기 (chunk size)

$$C^{(\ell)} := \frac{\max_{\ell'} C^{(\ell')}}{f_{\ell}}$$

와 연관된다. 즉 청크 크기는 가장 느린 블록의 청크 크기를 그 블록의빈도로 나눈 값이다. 입력 $x = \{x_1, \dots, x_T\}$ 가 주어지면 사슬의출력은 다음과 같이 계산된다 (명료함을 위해 정규화는 생략).

$$y_t = MLP^{(f_k)}(MLP^{(f_{k-1})}(\dots MLP^{(f_1)}(x_t))) \quad (1)$$

이 식은 입력이 고빈도 블록부터 저빈도 블록까지 MLP 사슬을 차례로 통과함을 뜻한다. 여기서 ℓ 번째 MLP 블록의 파라미터 $\theta^{(f_\ell)}$ 는 매 $C^{(\ell)}$ 스텝마다 업데이트된다. 즉 $\theta_{i+1}^{(f_\ell)} = \theta_i^{(f_\ell)} - e_{i,\ell}$ 이며,

$$e_{i,\ell} = \begin{cases} \sum_{t=i-C^{(\ell)}}^i \eta_t^{(\ell)} f(\theta_t^{(f_\ell)}; x_t) & \text{if } i \equiv 0 \pmod{C^{(\ell)}}, \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

이 식은 블록이 자기 체크 구간의 오차항을 누적했다가 체크 경계에서 한번에 파라미터에 반영함을 뜻한다. 여기서 $\eta_t^{(\ell)}$ 는 $\theta^{(f_\ell)}$ 에 대응하는 학습률이고, $f(\cdot)$ 는 신경망의 end-to-end 최적화에서 임의의 최적화기의 오차 성분 (예: 경사하강의 경우 $\nabla \mathcal{L}(\theta_t^{(f_\ell)}; x_t)$) 이다. CMS 의 형식화는 매우 광범위하며, 과제에 따라 목적함수 $\mathcal{L}(\cdot)$ 은 (Transformer 의 MLP 블록처럼) 바뀔 수 있다. 이러한 일반성과 Transformer 설계의 상위집합 (즉 MLP 블록이 1 개면 Transformer 와 동등) 이라는 성질때문에, 본문 전반에서 CMS 를 아키텍처의 기본 구성 블록으로 사용한다. 또한 명료함을 위해, 일반성을 잃지 않고 $C^{(\ell)}$ 이 $C^{(\ell-1)}$ 로 나누어 떨어진다고 가정한다. 식 (2) 는 중요한 해석을 제공한다. 파라미터 $\theta_t^{(f_\ell)}$ 는 자신의 문맥을 자신의 파라미터로 압축하는 역할을 하므로, 그 문맥의 추상 지식을 대표한다.

요약하면, 이 관점에서 시퀀스 모델 (어텐션이나 다른 기억 모듈, RNN 등) 은 모델의 **단기 기억** 으로 작동한다. 고빈도 업데이트가 옛 지식을 밀어내어 새 기억을 위한 공간을 만들기 때문이다. 반면 CMS 블록들은 기억모듈의 스펙트럼으로 작동하며, 앞쪽 블록 (고빈도) 은 더 단기적 기억, 뒤쪽 블록 (그리고 궁극적으로 빈도가 0 에 가까운 마지막 블록) 은 더 장기적 기억이다. 이 기억 시스템을 능동적으로 업데이트하면 CF 에 대한 저항성을 높일 수 있지만, **모든 모듈의 업데이트 주기가 어느 시점에 맞아 떨어지면 CF 가 발생할 수 있다.** 따라서 각 기억 블록을 업데이트하기 전에, 그 블록의 추상 지식을 더 안정적인 파라미터로 공고화하는 메커니즘이 결정적으로 필요하다.

2.3 문헌 속 수면 용어 (Sleep Terminology in the Literature)

인간의 수면 과정은 문헌 속 많은 연구 설계에 영감을 주었다. 특히 “꿈꾸기” 는 최근 경험/입력 데이터를 replay 하는 방법을 설계하도록 여러 과거 연구를 동기부여했다. 여러 연구는 모델을 long-horizon 과제에 더 강건하게 만드는 오프라인 과정을 설계했다. Lin et al. (2025) 은 다음 사용자 세션을 위해 과거 문맥을 요약하는 “sleep-time compute” 형태의 오프라인 자기 학습 과정을 제안했다.

우리가 아는 한, 기존 문헌 (수면 영감 연구 포함) 은 여전히 학습 · 시험 단계라는 전통적 구분에 굳게 묶여 있다. 지속/온라인 학습 설정에서도 모델은 파라미터를 업데이트하는 것 (학습) 과 성능을 평가하는 것 (시험) 을 오간다. 우리는 평생 적응을 위해서는 정적인 학습/시험 패러다임을 연속적 · 주기적인 “깨어 있음” 과 “수면” 의 생애 주기로 대체해야 한다고 주장한다. “깨어 있음” 단계에서 모델은 변화하는 입력 데이터와 능동적으로 상호작용하며 일시적 정보를 빠르게 습득하고, “수면” 단계에서 모델은 기억을 공고화하고 기존 지식을 내부적으로 처리한다.

3 수면 패러다임 (The Sleep Paradigm)

3.1 지속 학습에서의 학습 단계 (Learning Phases in Continual Learning)

앞서 논한 대로, 지속 학습자는 항상 데이터/경험으로부터 학습한다. 따라서 기계학습 모델의 생애 주기를 시험 시간과 학습 시간으로 나누는 전통적 구분은 지속 학습 설정에 직접 적용되지 않는다. 우리는 지속 학습자의 생애 주기의 두 핵심 단계로 “활동/깨어 있음” 시간과 “수면” 시간을 사용할 것을 제안한다. 활동/깨어 있음 시간에 지속 학습자는 새 입력 데이터를 받아 필요에 따라 처리한다. 수면 시간에는 입력 데이터를 최소한으로 (또는 전혀) 받지 않고 기억 공고화와 자기 개선을 위해 기존 지식을 처리하는 데 집중한다.

다만 기억 공고화 과정이 수면 단계에만 국한되는 것은 아니다. 실제로 Nested Learning (NL) 의 논의를 따르면 기억 공고화에는 두 가지 형태가 있다. (1) 온라인 공고화, (2) 오프라인 공고화다.

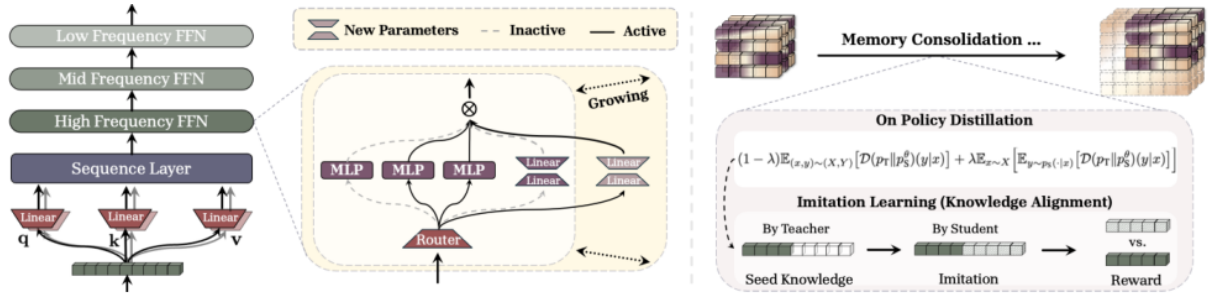


그림 2: 기억 공고화 개요. 모델은 스스로 파라미터 수를 늘려용량을 확장하고 (3.2 절), 그런 다음 knowledge seeding 을 사용해 지식 추상을고빈도 기억에서 저빈도 기억으로 전이한다 (3.3 절). 출처: 원저자, arXiv:2606.03979 Fig.2

활동 (깨어 있는) 단계에서 신경망의 온라인 공고화 온라인 공고화는깨어 있는/활동 단계에서 신경 학습 모듈 (예: 그림 1 의 Hope 아키텍처) 의 end-to-end 학습 과정을 통해 일어나며, 앞쪽 층 (고빈도의 빠른블록) 이 자신의 지식을 뒤쪽 층 (저빈도의 느린 블록) 으로 전이한다.

수면 단계에서 신경망의 오프라인 공고화 오프라인 공고화 단계에서모델은 수면 단계에 들어가 새 데이터 처리를 멈추지만, 스스로 생성한데이터를 사용해 최근 기억을 공고화 한다.

다음 절에서 수면 패러다임을 제시한다. 모델의 깨어 있는 (활동) 시간과달리, 모델은 어떤 외부 입력 데이터도 받지 않고 자기 개선, 과거 기억공고화, 지식 추상화에 내부 연산을 집중한다. 특히 수면 과정을 두 핵심단계로 나눈다. (1) 기억 공고화, (2) 자기 개선을 위한 꿈꾸기.

3.2 기억 공고화: 파라미터 확장 (Memory Consolidation: Parameter Expansion)

앞서 논한 대로 기억 공고화의 목표는 취약한 단기 기억을 더 방대하고안정적인 파라미터로 옮기는 것이다. CMS 형식화의 중요한 메시지 하나는**기억의 취약성/안정성이 상대적**이라는 것이다. 즉 각 기억 블록의 입장에서 보면, 더 높은 빈도의 다른 기억들은 더 단기적이고 더취약하다. 따라서 기억 공고화는 단순한 2 단계 과정이 아니라, 고빈도 기억에서저장된 지식을 더 안정적인 저빈도 파라미터로 반복적으로 전이하는**반복 연산**이다.

빠르게 업데이트되는 블록의 지식 (예: ICL) 을 잃지 않으려면, 그 파라미터를업데이트하기 **전**에 기억 공고화 스텝을 수행해야 한다. 따라서 청크길이 목록 $\{C^{(1)}, \dots, C^{(k)}\}$ 이 주어졌을 때, 수면 과정과그에 따른 기억 공고화는 모든 $b \in \mathbb{N}$ 에 대해 $\{C^{(1)} \times b, \dots, C^{(k)} \times b\}$ 스텝에서만 일어난다. MLP 블록들의 업데이트 빈도에 따라, 어떤 기억 모듈의 기억을 그 다음 기억블록으로 여러 번 공고화해야 할 수 있다. 예를 들어 업데이트 빈도 1K 인기억 뒤에 업데이트 빈도 10K 인 기억이 오는 경우, 느린 기억 블록이 한 번업데이트되기 전에 빠른 기억은 10 번 업데이트되므로, 느린 기억의 자체 업데이트 전에 빠른 기억에서 느린 기억으로 **10 번의 기억 공고화스텝**이 필요하다. 느린 기억으로 의 이 여러 공고화 스텝은, CF 때문에 LLM 의지속 학습 능력을 여는 데 결정적 병목이 될 수 있다. 이 현상은 모델의제한된 용량 (예: 파라미터 수) 의 본질적 결과로, 새 지식을 담기 위해파라미터를 덮어써야 하기 때문에 생긴다. 인간 뇌가 이 도전에 대응하는해법의 근간은 **신경가소성**, 즉 경험에 반응해 자신의 기능을 바꾸고새 연결을 형성하는 뇌의 본질적 능력이다. 이에 영감을 받아, 우리는 모델이새 연결을 형성하여 자신의 용량을 늘리게 하는 **효율적인 점진적파라미터 확장**을 제시한다.

일반성을 잃지 않고, MLP 블록들 $\{\text{MLP}^{(f_\ell)}(\cdot)\}_{\ell=1}^k$ 이 라우터 $\mathcal{R}^{(f_\ell)}$ 를 가진 **희소 전문가 혼합 (sparse mixture-of-experts, MoE)** 이라고 가정한다. 즉 각 $\text{MLP}^{(f_\ell)}(\cdot)$ 는 전문가 집합 $\{W^{(f_\ell),1}, \dots, W^{(f_\ell),s_\ell}\}$ 를 포함하며, $s_\ell \geq 1$ 은 사슬의 ℓ 번째 블록의 현재 전문가 수다. $(\ell^* - 1)$ 을 그 지식을 바로 다음의 더 안정적인 기억 모듈 (인덱스 ℓ^*) 로 공고화하려는 기억 (또는 MLP) 의 인덱스라 하자. $\text{MLP}^{(f_{\ell^*})}(\cdot)$ 에서 전이된 지식과 기존에 저장된지식의 간섭을 피하기 위해, 그 파라미터 집합에 **새로운 저랭크전문가**를 추가한다. 즉 다음으로 파라미터화된 저랭크 MLP 를 전문가 집합에추가한다.

$$\mathbf{A}^{(f_{\ell^*}), s_{\ell^*}+1} \in \mathbb{R}^{d \times d_{low}}, \quad \mathbf{B}^{(f_{\ell^*}), s_{\ell^*}+1} \in \mathbb{R}^{d_{low} \times d}, \quad (d_{low} \ll d)$$

이 저랭크 쌍은 좁은 병목 차원 d_{low} 를 거치는 새 전문가를 뜻한다. 이 새 파라미터들은 $\text{MLP}^{(f_{\ell^*-1})}(\cdot)$ 에서 새로전이되는 지식을 저장하도록 할당된다. 이 과정으로 매 수면 시간 후에는 일부 층들의 파라미터가 늘어난다.

3.3 Knowledge Seeding 을 통한 기억 공고화 (Memory Consolidation with Knowledge Seeding)

이 단계에서는 파라미터 $\theta^{(f_{\ell^*-1})}$ 를 가진 $\text{MLP}^{(f_{\ell^*-1})}(\cdot)$ 의 지식을 $\text{MLP}^{(f_{\ell^*})}(\cdot)$ 의 확장된 파라미터 집합으로전이하는 것이 목표다. LM_{θ} 를 파라미터확장 전의 언어 모델 상태로, $\text{LM}_{\theta_{exp}}$ 를 (i) 파라미터 확장과 (ii) 식 (2) 에 따른 $\theta^{(f_{\ell^*-1})}$ 의 업데이트가 이루어진 후의 언어 모델 상태로 둔다. 수면과 그에 따른 기억 공고화가 $\text{MLP}^{(f_{\ell^*-1})}(\cdot)$ 에 대해 일어나므로, 지난 스텝수는 C^{ℓ^*-1} 로 나누어 떨어지고 따라서 이 기억 블록은 업데이트된상태다. 우리는 기억 공고화 과정을 **증류 문제로** 모델링하여, 모델의 더 작은 상태 LM_{θ} 에 저장된지식을 더 큰 변형 $\text{LM}_{\theta_{exp}}$ 로전이하고자 한다.

상향식 증류 (Knowledge Seeding) 교사로서 일련의 작은 모델 $S_1(\cdot), \dots, S_k(\cdot)$ 가 주어졌을 때, **Knowledge Seeding(KS)** 은 이 모델들의 지식을 더 큰 모델 $\mathcal{M}(\cdot)$ 로 전이하는 것을 목표로 한다.

KS 형식화를 바탕으로 우리는 **self-Knowledge Seeding(SKS)** 을제시한다. 여기서는 (앞서 논한 대로) 더 작은 버전의 모델이 더 큰 버전의모델로 지식을 증류한다. 이 증류 과정에는 두 가지 결정적 도전이 있다. (1) 전통적 경우와 달리 **학생이 교사보다 용량과 표현력이 더 크다**. 따라서 교사가 생성한 고정 데이터셋 (예: Kim et al. 2016 의 sequence-level KD) 으로 학생을 학습하면 학생 파라미터를 최적으로 활용하지 못할 수 있다. (2) 모델이 수면 단계에 있어 **외부 정보/데이터셋 접근이 제한된다**. 따라서 Hinton et al. (2015) 같은 대중적 방법은 적용할수 없다. 이 도전을 극복하기 위해 우리는 on-policy 학생 생성 데이터와교사 생성 데이터의 혼합을 허용하는 **Generalized Knowledge Distillation(GKD, Agarwal et al. 2024)** 위에서 imitation learning 에기반한 새 증류 과정을 제시한다.

앞서 논한 대로, 기억 공고화 스텝은 단순히 원 데이터를 replay 하는 것이아니라, 깨어 있는 (활동) 스텝 동안 습득한 지식의 추상을탐색·추출해야 한다. 이를 위해 knowledge seeding 은 두 주요단계를 갖는다. (1) 학생이 스스로 생성한 시퀀스에 대해 교사의 logit 으로부터 토큰별 피드백을 받는 증류 과정, (2) 학생이 교사의 샘플링된출력을 기억하도록 강제하여, 증류된 지식을 보존하면서 둘의 샘플링 과정을정렬하는 RL 기반 imitation learning 방법이다.

먼저 교사 모델 LM_{θ} 에서 샘플링하여데이터셋 $\mathcal{D}$ 를 구성한다. 다음으로 GKD 를 따라 on-policy 증류목적함수를 다음과 같이 정의한다.

$$\mathcal{L}(\theta, \theta_{exp}) = (1 - \lambda) \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{F}(\text{LM}_{\theta} \parallel \text{LM}_{\theta_{exp}})(y|x)] + \lambda \mathbb{E}_{x \sim \mathcal{D}} [\mathbb{E}_{y \sim \text{LM}_{\theta_{exp}}(\cdot|x)} [\mathcal{F}(\text{LM}_{\theta} \parallel \text{LM}_{\theta_{exp}})(y|x)]]$$

이 식은 고정 교사 데이터에 대한 발산항과, 학생이 스스로 생성한 rollout 에대한 발산항을 λ 로 섞은 것을 뜻한다. 여기서 $\mathcal{F}(\text{LM}_{\theta} \parallel \text{LM}_{\theta_{exp}})(y|x)$ 는교사 (LM_{θ}) 와학생 ($\text{LM}_{\theta_{exp}}$) 출력 분포 간의발산이고, $\lambda \in [0, 1]$ 은 on-policy 학생 생성 출력의 비율을조절한다. 이 최적화 과정에서 우리는 **학생의 샘플링 분포를 통해역전과하지 않는다**. 이는 학습 안정성과 속도에 도움이 된다. 또한 학생모델의 모든 파라미터를 동결하고 확장된 파라미터만 업데이트한다. 이로써전이된 지식이 옛 지식과 간섭하여 CF 를 일으키지 않도록 보장한다.

Learning to Imitate(모방 학습). 위 증류 과정은 학생의 새파라미터가 저빈도 기억에 부호화된 지식을 저장하도록 보장한다. 그러나우리는 학생이 지식에 접근할 수 있음에도 **그것을 사용하는 법은배우지 못하여** 교사의 샘플링과 성능을 약하게만 흉내 내는 것을 관찰했다. 이를 위해 우리는 위 증류 과정을 RL 로 더 개선하여, 모델이 교사의 샘플링을모방하는 법을 가르친다. 교사가 생성한 데이터 (꿈) 집합 $\mathcal{D}_T = \{d^{(1)}, \dots, d^{(n)}\}$ 이 주어졌을 때, **Learning to Imitate(LTI)** 과정은 먼저 각 $d^{(i)}$ 에서접두부 (prefix) 를 무작위로 샘플링하고 학생 모델에 그 이어질 부분을완성하도록 요청한다. 학생의 응답 $\hat{d}^{(i)}$ 가 주어지면 할당되는보상은 다음과 같다.

$$r(\hat{d}^{(i)}; d^{(i)}; \text{LM}_{\theta_{exp}}) = \gamma \times r_{sem}(\hat{d}^{(i)}; d^{(i)}; \text{LM}_{\theta_{exp}}) + (1 - \gamma) \times r_{abs}(\hat{d}^{(i)}; d^{(i)}; \text{LM}_{\theta_{exp}}) \quad (3)$$

이 식은 의미 보상과 절대 (토큰 수준) 보상을 γ 로 섞은 것을 뜻한다. 여기서 $r_{sem}(\cdot)$ 은 의미적 유사도에, $r_{abs}(\cdot)$ 는 절대적 토큰 수준 유사도에 기반해 보상을 부여한다. 의미적 유사도의 경우, 동결된 보상 모델을 사용하여 $\hat{d}^{(i)}$ 와 $d^{(i)}$ 의 의미가 같으면 (각각 다르면) 1(각각 0)의 보상을 준다. 반면 절대 보상은 두 시퀀스의 Levenshtein 거리 ($z(\cdot, \cdot)$ 로 표기)에 기반하여 다음과 같이 정의된다.

$$r_{abs}(\hat{d}^{(i)}; d^{(i)}; \text{LM}_{\theta_{exp}}) = \begin{cases} 1 - \frac{z(\hat{d}^{(i)}, d^{(i)})}{\max\{|\hat{d}^{(i)}|, |d^{(i)}|\}} & \text{if } z(\hat{d}^{(i)}, d^{(i)}) \leq z_0, \\ 0 & \text{otherwise,} \end{cases} \quad (4)$$

여기서 z_0 은 유사도 임계값이다. 이 식은 편집 거리가 임계값 이하일때만 정규화된 유사도를 보상으로 주고, 그렇지 않으면 0을 준다는 뜻이다. 위 LTI 과정을 on-policy 증류에 통합하면 knowledge seeding(KS) 목적함수는 다음과 같이 정의된다.

$$\mathcal{L}_{KS}(\theta, \theta_{exp}) = \mathbb{E}_{x \sim \mathcal{D}} \left[(1 - \alpha) \mathbb{E}_{y \sim \text{LM}_{\theta_{exp}}(\cdot|x)} [r(y)] - \alpha \mathbb{E}_{y \sim \text{LM}_{\theta_{exp}}(\cdot|x)} \mathcal{D}(\text{LM}_{\theta} \| \text{LM}_{\theta_{exp}})(y|x) \right]$$

이 식은 LTI 보상항과 증류 (발산) 항을 α 로 균형 잡은 것을 뜻한다. 여기서 $\alpha \in [0, 1]$ 은 LTI 목적 대비 증류의 강도를 조절한다. 이 목적함수에 기반해 모델의 새로 확장된 파라미터를 업데이트하여 고빈도기억의 지식을 저빈도 기억 블록으로 공고화한다. 이제 $\text{MLP}^{(f_{\ell^*-1})}(\cdot)$ 의 기억이 $\text{MLP}^{(f_{\ell^*})}(\cdot)$ 에 공고화되었으므로, 과거 수면기간에 $\text{MLP}^{(f_{\ell^*-1})}(\cdot)$ 에 추가된 모든 저랭크파라미터를 리셋(reset)하여 그 용량을 미래를 위해 다시 확보한다. 이 스텝은 뇌가 불필요하거나 중복된 연결을 가지쳐 효율과 성능을 높이는 인간 뇌의 시냅스 가지치기 (synaptic pruning)와 유사한 절차로 해석할 수 있다.

구현에 관한 노트. 성장하는 희소 모듈을 구현하는 것은, 텐서의 차원을 직접 바꿔야 한다면 극도로 어려울 수 있다. 대안으로, 그 파라미터들을 처음부터 모델에 두되, 어떤 수면 단계에서 최초로 활성화되기 전까지는 순전파·역전파에서 마스킹할 수 있다. 흥미롭게도 이는 인간 뇌에 대한 우리의 이해와도 맞아떨어진다. 뇌는 (크지만) 고정된 용량을 가지며 시간이 지나며 새 구성요소가 추가되지 않는다. 대신 뇌 영역사이의 새 연결이 생애에 걸쳐 형성되어, 새 뉴런의 활성화를 잠금 해제하고 새 과제를 학습할 더 큰 가소성을 낳는다.

3.4 꿈꾸기: 자기 수정 과정 (Dreaming: A Self-Modifying Process)

이전 단계는 고빈도 파라미터를 동결하고 그 지식을 저빈도 기억으로 증류하는 것으로, 기억 공고화를 담당하는 인간의 서파 수면 단계 (NREM)와 유사하게 작동한다. 그러나 REM 단계에서 뇌는 (깨어 있을 때에 필적할 만큼) 매우 활발하며, 꿈꾸기를 통해 스스로를 수정하고 새로 형성된 시냅스를 강화하는 것을 목표로 한다. 이에 영감을 받아, 우리는 시간이 지나며 스스로를 개선하도록 돕는 꿈 (합성 데이터)을 생성하는 법을 학습하는 꿈꾸기 과정을 설계하고자 한다.

실제로는 자기 개선을 위한 어떤 합성 데이터 생성 과정도 이 단계에 통합될 수 있다. 다만 결정적 고려사항은, 지속 학습 설정에서 자기 개선을 반복적으로 적용할 때 CF가 발생할 위험이다. 우리의 평가에서는 기억 공고화 후 꿈꾸기라는 2단계 설계가 CF에 더 강건함을 보인다. 개념 증명으로 우리는 SEAL(Zweiger et al. 2025) 위에서 구축하되, 이를 우리의 수면패러다임에 통합하기 위한 세 가지 도전을 해결한다. (1) SEAL의 내부루프에서 지도 파인튜닝(SFT) 비용 때문에 적은 수의 self-edit(우리 용어로 꿈)에 국한된다. (2) 수면 기간의 반복적 자기 개선이 원인이 되는 잠재적 CF. (3) 샘플링 과정이 모델의 기존 지식 공간에서만 샘플링하는데, 꿈꾸기의 핵심 역할 중 하나는 **기억의 새로운 합성 (novel synthesis)**을 탐색하는 것이다.

샘플링된 과제 (C, τ)가 주어졌을 때— C 는 과제 관련 정보를 담은 문맥, $\tau(\cdot)$ 는 하위 평가에서 성능을 재는 척도—우리의 “꿈꾸기” 과정은 C 를 문맥에 둔 채 $m \geq 1$ 개의 꿈을 생성하는 것에서 시작한다. 샘플링 과정에서 MoE 블록의 각 라우터는 추가로 무작위전문가 하나를 더 선택하여, 무관한 무작위 지식을 꿈꾸기에 주입한다. 이로써 모델의 시야에서 숨겨진 근본 패턴을 학습한다. 이 스텝에서는 $\{\text{DREAM}^{(i)}\}_{i=1}^m \sim \text{LM}_{\theta}(\cdot|C)$ 로 둔다. 다음으로, 생성된 꿈 중 일부를 기각하고 모델 성능

개선 잠재력이 가장 큰 표본만 남긴다. 이를 위해 gradient 기반 데이터 선택 문헌에서 영감을 받아, 각 꿈 $\text{DREAM}^{(i)}$ 에 중요도 점수 $\omega^{(i)}$ 를 부여하고, 다양성 유지를 위해 중요도 점수 상위 Top- k 꿈과 b 개의 무작위 표본을 선택한다. 언어 모델링 목적 함수 $\mathcal{L}_{SFT}(\cdot)$ 가 주어졌을 때, $\text{DREAM}^{(i)}$ 의 중요도 점수 $g_{DR}^{(i)}$ 를 목적 함수의 gradient 로 정의한다.

$$g_{DR}^{(i)} = \nabla_{\theta} \mathcal{L}_{SFT}(\text{DREAM}^{(i)}, \theta)$$

이 식은 한 꿈이 모델을 얼마나 크게 바꿀지를 gradient 크기로 제어 중요도로 삼는다는 뜻이다. 위 과정으로 선택된 모든 꿈의 집합을 D 라 하자. 각 $\text{DREAM}^{(i)} \in D$ 에 대해, 모델의 고립된 인스턴스를 하나 두고 LoRA 를 사용한 지도 파인튜닝으로 그 파라미터를 업데이트한다.

$$\theta'^{(i)} \leftarrow \text{SFT}(\theta^{(i)}, \text{DREAM}^{(i)})$$

새로 파인튜닝된 모델이 주어지면, SEAL 을 따라 $\text{LM}_{\theta'^{(i)}}$ 의 성능이 $\text{LM}_{\theta^{(i)}}$ 대비 개선되었는지에 기반해 $\text{DREAM}^{(i)}$ 의 생성에 보상을 준다.

$$r(\text{DREAM}^{(i)}, \tau(\cdot), \text{LM}_{\theta^{(i)}}) = \begin{cases} 1 & \text{개선되면,} \\ 0 & \text{그 외.} \end{cases} \quad (5)$$

이 식은 꿈이 다운스트림 성능을 개선했을 때만 그 꿈의 생성을 보상하는 이진 보상을 뜻한다. 우리는 SEAL 을 따라 위 과정을 최적화하기 위해 **ReST^{EM} 알고리즘** (Singh et al. 2024a) 을 사용한다.

4 실험 결과 (Empirical Results)

실험에서는 수면의 각 단계와 모든 단계를 함께 사용한 효과를 연구한다. 첫절에서는 지속 학습 능력과 long-context 이해를 높이도록 설계된 기억 공고화에 초점을 둔다. 추가 실험 결과는 부록 B 를 참고하라.

4.1 기억 공고화의 효과 (The Effect of Memory Consolidation)

기억 공고화 과정의 주요 목표 하나는 long-context 이해를 강화하면서 지속 학습을 개선하는 것이다. 이 절에서는 (자기 개선 없이) 기억 공고화 단계만을 지속 학습 · long-context 벤치마크에서 평가한다.

클래스 증분 학습 (Class Incremental Learning). 먼저 CLINC, Banking, DBpedia 세 데이터셋에서의 클래스 증분 학습에 초점을 둔다. 백본으로 Llama-3B 와 Llama3-8B 를 사용한다. Hope 는 우리의 기억 공고화 메커니즘으로 증강되며, on-policy 자기 증류를 통해 추상을 개선한다. Momeni et al. (2025) 을 따라 ICL(수면 없이 동일한 지속 사전 학습 과정), Elastic Weight Consolidation(EWC), 외부 학습자를 활용한 In-context Continual Learning(InCA) 과 비교한다. 또한 명시적 증류 과정이 없는 다층 in-context 업데이트 기준선으로 Hope 도 포함한다. 그림 3 의 결과는 Hope 가 외부 학습자 (InCA) 와 정규화 (EWC) 접근을 포함해 모든 데이터셋에서 가장 좋은 성능을 낼 수 보인다. ICL 대비 이득은 프롬프트 수준 적응을 공고화를 통해 지속적 파라미터 기억으로 전환하는 데서 온다. Hope 대비로는, 명시적 자기 증류가 반복적 in-context 업데이트만 사용하는 것보다 더 나은 추상을 낳는다.

레벨 수 (#수면 단계) 가 ICL 에 미치는 효과. Hope 의 공고화 스케줄이 ICL 과 long-context 이해에 미치는 영향을 더 잘 분리하기 위해, long-context 에서의 질의 응답과 다중 키 검색을 평가한

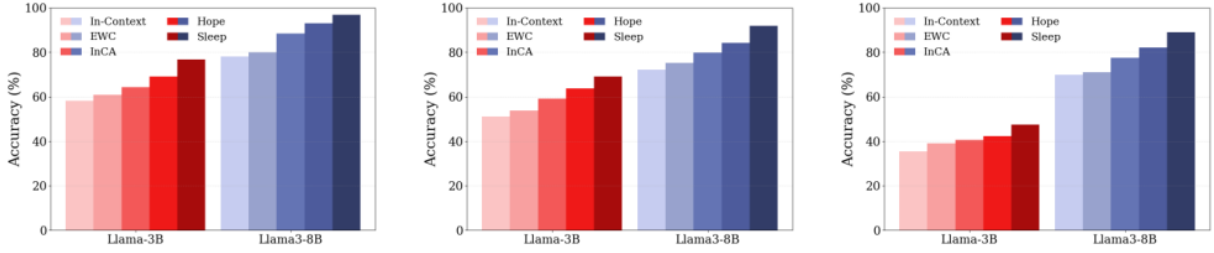


그림 3: 텍스트 분류에서의 클래스 증분 학습을 (좌) CLINC, (중) Banking, (우) DBpedia 데이터셋에서 평가. Hope 아키텍처가 다른 지속 학습접근을 일관되게 능가하며 가장 높은 정확도를 달성한다. 출처: 원저자, arXiv:2606.03979 Fig.3

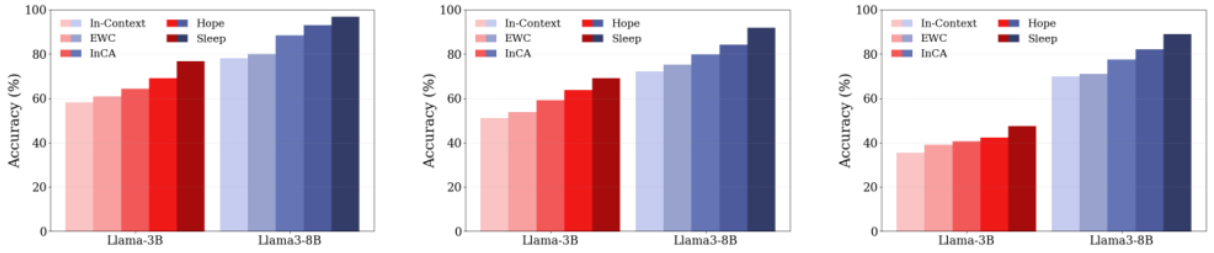


Figure 3: Class-incremental learning for text classification is evaluated on the (Left) CLINC dataset (Larson et al. 2019), (Middle) Banking dataset (Casanueva et al. 2020), and (Right) DBpedia dataset (Auer et al. 2007). The HOPE architecture consistently outperforms other continual learning approaches, achieving the highest accuracy.

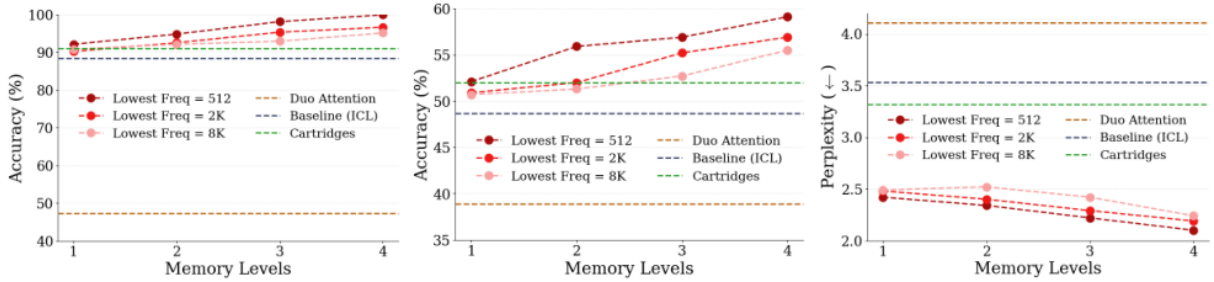


그림 4: 기억 레벨이 ICL 성능에 미치는 효과. (좌) RULER의 MK-NIAH, (중) LongHealth, (우) QASPER. QASPER는 값이 낮을수록 성능이 좋다. 출처: 원저자, arXiv:2606.03979 Fig.4

다. LongHealth, QASPER, MK-NIAH 데이터셋을 사용한다. 기준선으로 ICL, DuoAttention, Cartridges를 사용한다. Hope 변형의 경우, 수면 스케줄을 (i) 사용하는 공고화 단계 수와 (ii) 가장 안정적인 기억의 지속성 (가장 낮은 공고화빈도로 조작)을 바꿔 변화시킨다. 직관적으로 더 낮은 빈도는 더 지속적이지만 덜 적응적인 장기 기억을 낳는다. 결과는 그림 4에 보고된다. 모든 과제에서 Hope는 ICL과 효율적 기준선 DuoAttention을 일관되게 능가하여, 수면 시간 공고화가 프롬프트만의 적응을 넘어 long-context 행동을 개선함을 보인다. 또한 Hope는 Cartridges를 능가한다. Cartridges가 보조 모델로 KV 표상을 압축해 효율을 높이는 반면, Hope는 수면 중 on-policy 자기 증류를 수행하여 새로 습득한 정보를 전이 가능한 파라미터 지식으로 공고화하고, 더 강건한 long-context 이해를 낳는다. Hope 변형들끼리 비교하면 두 가지 일관된 경향을 발견한다. (1) 공고화 단계 수를 늘리면 ICL과 long-context 이해가 개선되어, 수면이 더 나은 지식추상·압축을 가능케 함을 뒷받침한다 (더 적은 유효 파라미터로 더 많은 정보를 유지). (2) 가장 낮은 빈도를 높이면 성능이 떨어져, 가장 지속적인 기억을 더 적응적으로 만들면 유지력이 약해짐을 시사한다.

In-Context로 새 언어 학습. LLM은 지속 학습 설정에서 자주 실패한다. 여기서 모델은 이전에 습득한 지식을 덮어쓰지 않으면서 새 기술을 순차적으로 습득해야 한다. 이 도전을 연구하기 위해 Behrouz et al. (2025)이 도입한 과제를 따른다. 이는 사전학습 중 보지 못한 두 언어에 대한 번역 데이터셋인 MTOB(Kalamang)와 Manchu를 결합한 것이다. 즉 모델은 사전학습 중 보지 못한 두

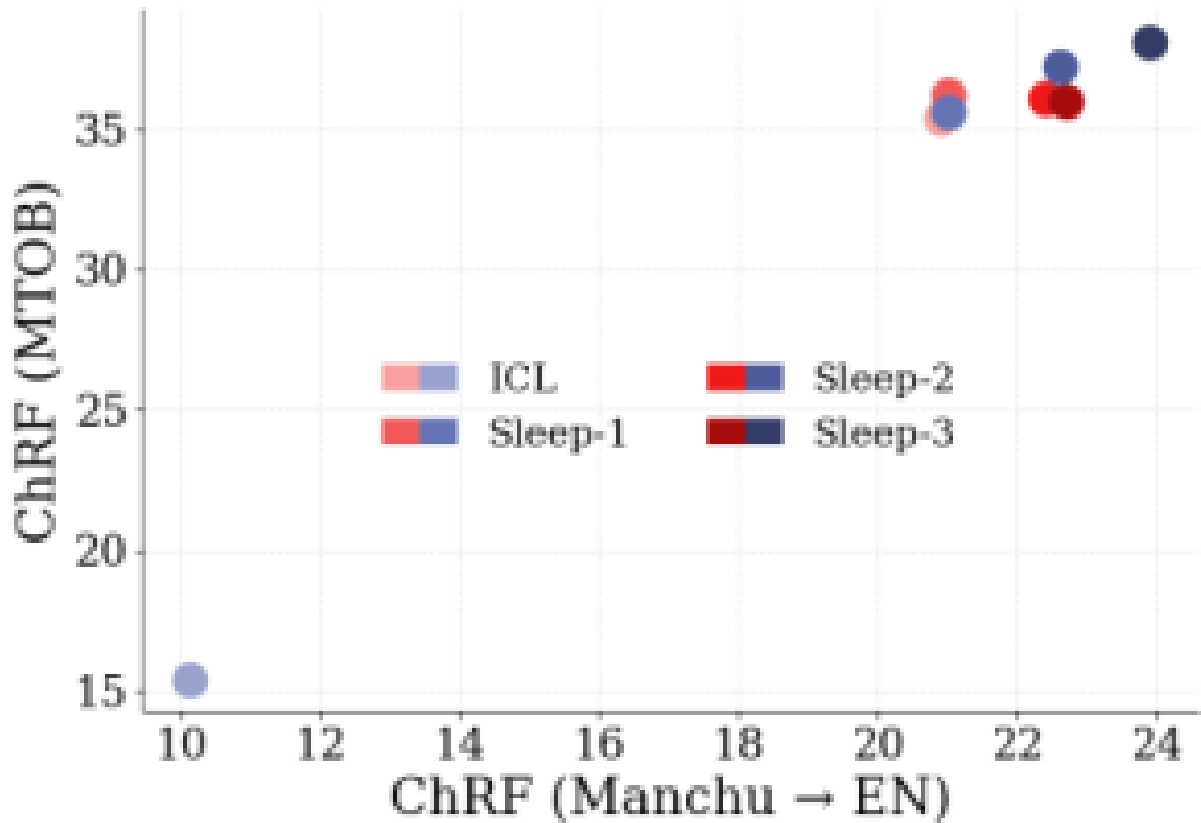


그림 5: 새 언어의 지속 번역 (CTNL) 과제. 빨간 점은 단일 언어로 학습할 때의 성능, 파란 점은 지속 학습 하의 성능을 보인다. 출처: 원저자, arXiv:2606.03979 Fig.5

언어에 in-context 로 노출되어 구절을 영어로 번역해야 한다. 두 설정을 고려한다. 각 언어를 독립적으로 학습·평가하는 경우와, 두 언어를 순차적으로 학습한 뒤 각 언어의 번역 성능을 평가하는 경우다. 표준 ICL 을, 수면 시간 공고화 단계수가 다른 Hope 변형 (Hope-1, Hope-2, Hope-3) 과 비교한다. 결과는 그림 5 에 있으며, 각 점은 Manchu → English(x 축) 와 Kalamang → English(y 축) 의 ChRF 점수를 보고한다. 단일 언어 설정에서 모든 Hope 변형은 ICL 과 같거나 그 이상이며, 공고화가 in-context 적응을 저해하지 않음을 나타낸다. 지속학습 설정에서 ICL 은 급격한 성능 하락을 보이며 대체로 사전학습 행동으로 되돌아가는 반면, Hope 는 그 이득을 상당 부분 유지한다. 성능은 공고화 단계가 늘수록 단조 개선되며, Hope-3 은 순차 노출에도 불구하고 단일 언어 성능을 거의 회복한다. 또 다른 기준선으로 Cartridges 와 SFT 도 평가했는데, 놀랍게도 두 방법 모두 적어도 한 언어에서 CF 를 겪어 적어도 한 과제에서는 ICL 보다도 약했다 (그림 5 에서 도표 바깥에 위치).

BABILong. Hope(Sleep) 를 BABILong 벤치마크에서 평가하며, (1) 대형 모델 (GPT-4, GPT-4o-mini), (2) RAG 를 붙인 중규모 Llama-8B, (3) RMT·ARMT·Titans 등 최첨단 소형 long-context 모델과 비교한다. 자세한 논의는 B.2 절에 있다. 요약하면, Hope 는 1000 만 (10M) 토큰 까지 확장하면서 거의 완벽한 점수를 달성한다.

추론을 위한 기억 공고화. 기억 공고화 단계의 또 다른 중요한 함의는 모델의 추론 능력 개선이다. 이 절에서는 수학 추론에 대한 효과를 평가하고, 기본 모델·SFT·GRPO 기준선과 비교한다. 결과는 표 2 에 보고된다. 우리의 기억 공고화 알고리즘은 기본모델의 추론 능력 개선에서 SFT 와 GRPO 보다 낫다.

표 2: 수학 추론 벤치마크에서 여러 방법의 성능. Qwen 모델의 여러 변형을 사용하고 average@16 을 보고한다.

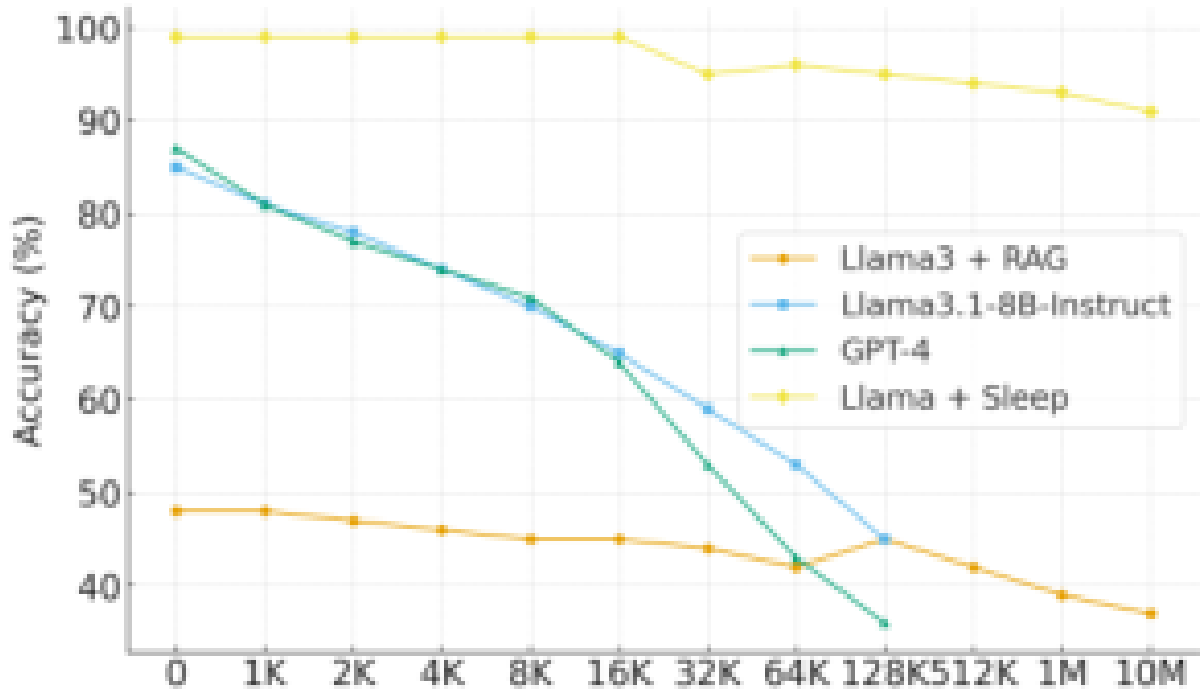


그림 6: BABILong 벤치마크 결과. 빨간 점은 파인튜닝된 모델, 파란점은 대규모 모델의 zero-shot 평가에 대응한다. 출처: 원저자, arXiv:2606.03979 Fig.6

방법	AIME-24	AIME-25	HMMT-25
Qwen3-1.7B			
Base (Instruct)	49.8	34.5	25.7
SFT	47.3	36.1	22.9
GRPO	51.0	38.6	26.1
OPSD	51.6	40.0	28.1
Sleep	53.2	40.2	29.3
Qwen3-8B			
Base (Instruct)	73.8	68.1	42.4
SFT	75.5	66.4	43.7
GRPO	76.4	68.1	44.9
OPSD	76.6	67.4	45.1
Sleep	79.2	69.0	46.1

지식 통합 (Knowledge Incorporation). 다음으로 자기 개선까지 허용하는 Sleep의 전체 설계에 초점을 둔다. 이 과제에서 모델은 통합된 사실에 관한 질문에 답할 수 있어야 한다. 공정한 비교를 위해 모델과 파라미터 선택을 포함해 Zweiger et al. (2025)의 실험 설정을 따른다. SQuAD 데이터셋의 새 사실 정보를 통합하는 것으로 평가한다. 기준선으로 (i) 개선이나 지문 접근이 없는 기본 모델, (ii) 꿈꾸기 없는 파인튜닝 모델, (iii) RL과 자기 적응을 가진 SEAL, (iv) 2단계 기억 시스템의 Transformer 기반 아키텍처, (v) 4단계 기억 시스템을 사용한다. 표 3은 단일지문 ($n = 1$)과 지속 사전학습 (CPT, $n = 200$) 설정 모두에서 문맥없는 (no-context) SQuAD 평균 정확도를 요약한다. 우리의 수면 과정은 다른 설정 및 SEAL 같은 최첨단 방법 가운데 최고 결과를 달성한다. 이를 (1) 모델이 지식을 더 효과적으로 저장하게 하는 기억 공고화 스텝, (2) 3.4절에서 논한 SEAL에서의 개선에 기인한다고 본다. CPT 체제에서 모델은 단일 지속 사전학습 실행 동안 $n = 200$ 개의 지문에 노출되고 연관된 974개 질문 전체로 평가된다. 각 지문마다 다섯 개의 꿈을 샘플링해 하나의 집계된 합성 데이터셋으로 결합하여 학습한다. 수면 과정이 최고 성능을 달성한다.

표 3: 지문 설정별 지식 통합 성능 (문맥 없는 SQuAD 정확도).

방법	단일 지문 ($n = 1$)	지속 사전학습 ($n = 200$)
Base model	31.9	31.9
꿈꾸기 없는 파인튜닝 모델	33.4	32.0
SEAL	46.7	43.2
Sleep (Transformer)	48.1	44.3
Sleep (Transformer + 4 단계)	48.9	46.2
gradient 기반 선택 제거	47.1	45.2
무작위 전문가 제거	48.0	44.7
꿈꾸기 전체 제거	35.7	36.2

Few-Shot 학습. 선행 연구의 few-shot ARC 실험 프로토콜을 따르되 우리의 수면 패러다임에 맞게 조정한다. 백본으로 Llama-3.2-1B 를 사용한다. 표준 구성에서 풀 수 없는 과제를 피하기 위해 데이터 일부를 필터링하여, 학습용 11 개 과제와 평가용 held-out 8 개 과제를 얻는다. 이 설정 (표 4) 에서 Sleep 은 80%의 성공률을 달성하여 다른 방법보다 높다.

표 4: Few-shot 추상 추론 (성공률 %).

방법	성공률 (%)
ICL	0
TTT	10
SEAL	72.5
Sleep	80

4.2 설계 선택에 대한 소거 실험 (Ablations on the Design Choices)

수학 추론에서 기억 공고화 과정의 설계 선택을 소거한다. 결과는 표 1 에보고된다. 모든 구성요소가 우리 방법의 성능에 긍정적으로 기여한다. 자기개선에 대한 설계 선택도 표 3 에서 추가로 소거한다.

표 1: 수학 추론 벤치마크에서 여러 방법의 성능 (소거 실험, Qwen3-8B, average@16).

방법	AIME-24	AIME-25	HMMT-25
Sleep	79.2	69.0	46.1
– Imitation Learning	76.8	67.9	45.0
– Semantic Reward	78.9	69.2	44.5
– w/o Expansion (확장 없음)	78.2	67.9	44.9
OPSD	76.6	67.4	45.1
OPSD + Expansion	77.9	68.2	45.9

5 결론 (Conclusion)

본 연구에서 우리는 LLM 을 위한 Sleep 패러다임을 도입했다. 이는 (i) 단기 ·in-context 지식을 저빈도의 장기 파라미터로 전이하는상향식 증류인 **knowledge seeding** 과, (ii) 간섭을 제어하면서 역량을 개선하는 스스로 생성한 학습인 **dreaming** 으로 구성된다. long-context 이해, 지식 통합, few-shot 추론, 지속 학습에 걸친 실험결과에서 Sleep 은 일관된 이득을 낸다.

부록 A 관련 연구 (Related Work)

A.1 파라미터 효율적 적응과 합성 (Parameter-Efficient Adaptation and Composition)

파라미터 효율적 파인튜닝 (PEFT) 은 백본을 동결한 채 최소한의 보조파라미터만 최적화하여 LLM 을 특정 과제에 적응시킨다. **저랭크·점두부 적응**. 대표적 방법으로는 선형 투영에학습 가능한 저랭크 행렬을 주입하는 LoRA 와, 학습 가능한 가상 토큰으로 연산을 유도하는 prefix/prompt-tuning 이 있다. 최근 변형들은 수렴을 가속하기 위해 초기화를 최적화하거나 (예: PiSSA), 이 메커니즘을 문맥→파라미터 증류로 확장한다 (Cartridges). **합성과 라우팅**. 단일과제 적응을 넘어, 여러 어댑터를 합성하는 연구가 늘고 있다. ICL 을 통한가중 합성이나 입력별로 관련 LoRA 모듈을 동적으로 선택하는 검색 기반라우팅 등이 있으며, 고급 구성은 MoE 아키텍처를 활용해 특정 저랭크 경로를 활성화하거나 과제에 따라 어댑터 파라미터를 병합한다.

A.2 지식 주입, 증류, 자기 개선 (Knowledge Injection, Distillation, and Self-Improvement)

파라미터 지식 주입. 추론 시 검색 의존을 줄이기 위해, 최근연구는 외부 지식을 모델 파라미터에 직접 주입한다. 문서별 LoRA 어댑터를부여하는 Parametric RAG 부터, 학생 모델이 교사 생성 QA 쌍이나 합성대화로부터 학습하는 prompt distillation 까지 다양하다. Cartridges 는 자기학습 (self-study) 목적으로 재사용 가능한 “KV cache 어댑터” 를 사전학습하여 PEFT 와 증류를 잇고, 서빙 비용을 줄이면서 ICL 품질을 달성한다. **자기 개선과 메타학습**. 증류를 보완하는 것으로, 모델이 생성한신호를 자기 개선에 사용하는 방법이 있다. 검증 가능한 보상에 대한 RL 로추론을 강화하거나 자기 보상 메커니즘을 쓰는 것이 포함된다. SEAL 같은메타학습 프레임워크는 적응 전략 자체를 최적화 (self-edit 생성 방법을학습) 하여 이를 확장하며, 자기 참조 학습 (self-referential learning) 의넓은 원리에 기댄다. 이 방법들은 감독을 확장하기 위해 고품질 합성 데이터생성에 크게 의존한다.

최근 동시대 연구 그룹은 자기 증류 과정에 on-policy distillation 을사용하거나 지속 학습에 사용할 것을 제안했다. 이 방법들보다 우리 연구가더 오래되었다는 사실에 더해, Sleep 은 근본적으로 다른 메시지를 전한다. (1) Sleep 은 파라미터를 잠금 해제하는 **자기 자신의 상향식 증류**를사용한다. (2) Sleep 은 on-policy 방법과 RL 방법을 결합한 **Generalized Distillation** 에 기반한다. (3) Sleep 은 지식이 서로다른 업데이트 빈도의 모듈에 저장되는 **주기적 과정**을 제안한다.

A.3 효율적 문맥 처리 (Efficient Context Processing)

long-context 처리의 메모리 병목을 다루는 것은 KV cache 를 압축하거나어텐션 아키텍처를 수정하는 것을 포함한다. **프롬프트·캐시 압축**. 메모리 사용을 줄이는 접근은 두범주로 나뉜다. 프롬프트 압축은 토큰 필터링 (하드 토큰) 이나 오토인코더로학습한 압축된 소프트 토큰 임베딩으로 입력을 짧게 만든다. KV cache 압축은런타임에 작동하며, 비필수 키를 버리는 축출 정책이나유사도·어텐션 밀도에 기반한 토큰 병합을 쓴다. KV cache 의저랭크 투영도 높은 압축률에서 성능 유지에 유망함을 보였다. **아키텍처 혁신**. MHA 에 대한 구조적 변화로는 KV 헤드를 공유해메모리 대역폭을 줄이는 MQA/GQA, softmax 를 제거해 시퀀스길이와 무관한 고정 크기 상태 ($K^T V$) 를 얻는 선형화 기법이 있다. Titans, TTT 같은 최근의 “learning to compress” 아키텍처는 고정 크기 기억 객체에 대한 gradient 기반 업데이트로 무한 문맥을 다룬다. 끝으로 MemGPT 같은 오케스트레이션 시스템은 아키텍처 수정 없이 가상 메모리 페이지징으로문맥을 관리한다. 토큰 공간 압축과 유사한 방향으로, Lin et al. (2025) 은 sleep-time compute 를 제시한다. 이름은 비슷하지만 이 방법은 우리 연구와근본적으로 다르다. 이 방법이 모델이 유휴 상태일 때 사용자와의 상호작용의좋은 요약을 텍스트 공간에서 찾는 반면, 우리 제안은 증류를 사용해 텍스트데이터 지식을 파라미터 가중치 업데이트 형태로 전이한다.

A.4 On-Policy Distillation 에 관한 최근동시대 및 후속 연구 (Recent Concurrent and/or Later Work on On-Policy Distillation)

On-Policy Self-Distillation(OPSD) 에 관한동시대·후속 연구. 본 연구의 초기 버전과 동시대에 또는이후로, on-policy self-distillation(OPSD) 을 탐구하는 문헌이 빠르게 늘었다. OPSD 에서는 같은 모델이 서로 다른 조건화 문맥 하에서 교사와 학생역할을 모두 맡고, 학생은 교사에 대한 토큰별 발산으로 자신의 rollout 에 대해 감독된다. 정전적 (canonical) 사례는 OPSD 로, 검증된 추론 자취 (trace) 로 교사를 조건화하고 학생 rollout 에 대한 토큰별 reverse-KL 을 최소화하여 GRPO 와 off-policy 증류를 모두 능가하는 수학 추론 이득을 낸다. 이후 일련의 후속 연구는 특권 문맥의 형태를 다양화한다. On-Policy Context Distillation 은 일시적 in-context 정보 (과거 해법 자취 나 최적화된 시스템프롬프트) 로 교사를 조건화하여 학생이 그 정보를 파라미터로 내재화하게 한다. GATES 는 문서 기반 특권 문맥과, 여러 튜터 자취를 샘플링해 그 합의로 학습을 게이팅하는 합의 게이팅 메커니즘을 사용한다. SD-Zero 는 학생 응답과 이진 보상으로 “reviser” 를 조건화한 뒤 이를 생성기로 되증류하여 희소한결과 보상을 조밀한 토큰 수준 감독으로 바꾼다. COPSD 는 교사에 영어 번역과참조 해법을 특권 교차언어 문맥으로 주어 저자원 언어로 추론 행동을 전이한다. Apple 의 “embarrassingly simple” 자기 증류는 이 스펙트럼의 극단에 있으며, 명시적 교사를 아예 없애고 서로 다른 디코딩 구성 하에서 모델 자신의 표본으로 SFT 하여, 이 최소 레시피조차 코드 생성을 유의미하게 개선함을 보인다.

지속·경험·온라인 적응. 두 번째 군집은 자기 증류를 지속·경험·온라인 개선에 적용한다. SDFT 는 시연 조건화 모델을 on-policy 교사로 사용해 시연으로부터의 지속학습을 명시적으로 겨냥하며 표준 SFT 대비 CF 를 줄인다. OEL 은 문맥 증류와배포를 결합하여, 상호작용 궤적에서 전이 가능한 경험적 지식을 추출해 on-policy context distillation 으로 파라미터에 공고화하고, 두 단계를 반복하여 달린 온라인 학습 루프를 만든다. π -Play 는 이 아이디어를 다중 에이전트 self-play 체제로 더 밀어붙인다. 이 연구들은 LLM 이 배포 후정적이어서는 안 된다는 우리 동기와 밀접히 정렬되나, 지속·경험적 개선을 고정 용량 체크포인트에 대한사후학습 목적으로 정식화한다. 대조적으로 우리 Sleep 패러다임은 지속학습을 기억 시스템 문제로 다룬다. 새로 습득한 정보는 먼저 빠르고 취약한 기억 모듈에 표현된 뒤, Knowledge Seeding 을 통해 주기적으로 더 느리고안정적인 파라미터로 공고화되며, 점진적 용량 확장과 공고화 후 고빈도기억의 리셋이 수반된다.

응용 특화 OPSD 레시피. 세 번째 그룹은 OPSD 틀을 특정 배포체제에 맞춘다. CRISP/OPSDC 는 “간결하게” 지시 접두부교사를 조건화하고 학생 rollout 에 토큰별 reverse-KL 을 써서 엔트로피 붕괴없이 긴 chain-of-thought 를 압축한다. MTP 는 단일 온라인 자기 증류목적으로 사전학습된 next-token 예측 모델을 다중 토큰 예측기로 바꿔 GSM8K 에서 정확도 하락 <5%로 3 배 이상 추론 가속을 낸다. OPSDL 은추출된 관련 스패에 대한 모델 자신의 단문맥 행동을 장문맥 학생의 자기교사로 사용해 장문맥 생성을 공략한다. Skill-SD 는 다중 턴 에이전트를겨냥하여 완료된 궤적을 간결한 자연어 “skill” 로 요약해 교사만 조건화한다. MSD 는 OPSD 를 다국어 설정으로 옮겨 다국어 질의만으로 안전 행동을고자원에서 저자원 언어로 전이한다. 종합하면, 특권 조건화 하의 자기증류는 추론·효율·에이전트·다국어축에 걸쳐 일반적이고 유연한 사후학습 레시피지만, 각 경우 고정모델과 고정 응용에 대한 단일 목적 절차로 배포된다.

OPSD 의 한계와 Sleep 패러다임의 동기. 이 영역에서 OPSD 의 경험적성공에도 불구하고, 최근 분석은 그 실패 모드를 드러내기 시작했다. Kim et al. (2026) 은 수학 추론에서 풍부한 특권 정보로 교사를 조건화하면 모델의인식론적 언어화 (추론 중 불확실성 표현) 를 억제하여, 빠른 도메인 내 최적화를 가능케 하지만 심각한 분포 외 (OOD) 저하—Qwen3-8B, DeepSeek-Distill-Qwen-7B, Olmo3-7B-Instruct 에 걸쳐 최대 40% 하락—를대가로 치른다고 보인다. 여러 최근 OPSD 연구도 long-horizon 학습파이프라인에서 자기 증류를 순진하게 반복하면 특권 교사로부터의 정보누출과 학습 붕괴로 이어질 수 있다고 보고한다. 이 결과들은 자기증류만으로는 강건한 지속 개선에 충분치 않음을 시사한다. 고정 모델에 직접적용된 반복적 자기 개선 루프는 이전 역량을 침식하거나 추론을 불안정하게만들 수 있다. 우리의 2 단계 Sleep 프레임워크는 기억 공고화를 자기 개선과분리함으로써 이 우려를 직접 다룬다. Dreaming 이 모델을 추가로 수정하기전에, 공고화가 먼저 새로 습득한 지식을 새로 확장된 저빈도 파라미터에안정화하여, 반복적 자기 학습이 유용한 이전 역량을 덮어쓸 위험을 줄인다.

우리 연구의 위치. 우리 연구는 OPSD 문헌과 네 축에서 다르며, 이들이 함께 Sleep 패러다임을 동기부여한다. (i) 기억 공고화로서의상향식 증류. 위 자기 증류 연구들은 모두 교사와 학생이 같

은 백본을공유하고 조건화 문맥에서만 다르다. 대조적으로 우리 Knowledge Seeding 스텝은 상향식 증류다. 더 작고 고빈도인 기억 모듈이 저빈도 기억 모듈의새로 확장된 더 큰 저랭크 전문가 집합으로 증류된다. 이는 CF를 샘플링분포가 아닌 불충분한 용량의 문제로 재구성하고, 고정 교사를 재조건화하기보다 공고화 스텝 사이의 점진적 파라미터 성장으로 대응한다. (ii) 기억 빈도의 연속체. 위 연구들이 증류를 평평한학생/교사 쌍으로 다루는 반면, 우리 방법은 엄격히 정렬된업데이트 빈도를 가진 기억 블록 사슬에서 작동하며 연속한 각 블록 쌍사이에 공고화를 수행한다. 우리가 아는 한 SDFT 만이 우리를 동기부여하는지속 학습 요건을 명시적으로 겨냥하나, 아키텍처 성장이나 계층적 기억 없이그렇게 한다. (iii) 공고화를 넘어선 2 단계 과정으로서의 수면. OPSD 문헌은 거의 전적으로 공고화 스텝에 초점을 둔다. 우리 프레임워크는 REM 수면과 유사한 Dreaming 단계를 추가로 포함하며, 여기서 모델은 합성데이터 커리큘럼을 생성하고 gradient 기반 중요도 점수로 표본을 가중하며 MoE 라우터를 활용해 통제된 새로움을 주입한다. (iv) imitation learning 으로 on-policy 증류를 증강하는 knowledge seeding. 위 연구들의 on-policy 증류 목적이 학생 rollout에 대한 토큰별 reverse-KL로 환원되는반면, 우리 Knowledge Seeding 목적은 GKD 방식 on-policy 증류를 교사샘플링 분포와의 의미·Levenshtein 수준 정렬을 함께 보상하는 RL 기반 Learning-to-Imitate 향으로 증강하여, 더 큰 학생이 교사의 지식을몰려버릴 뿐 아니라 교사가 그것을 사용하는 방식까지 모방하게한다.

부록 B 추가 실험 결과 및 세부 사항 (Additional Experimental Results and Details)

모든 실험에서 원래 벤치마크의 설정 (각 절에 인용)을 따른다. 지면 절약과정보 중복 방지를 위해 모델 세부는 해당 연구들을 참조한다. 우리설계에서는 추가 파라미터로 차원 64의 MLP 블록 5개를 사용하고활성 파라미터 수는 기본 모델 (8B 또는 3B)과 동일하게 유지한다.

B.1 데이터셋 (Datasets)

클래스 증분 학습. 세 데이터셋에 초점을 둔다. - **CLINC:** CLINC150은 과제 지향 대화용 다중 도메인 의도 분류 벤치마크로, 범위내 (in-scope) 의도 예측과 범위 외 (OOS) 탐지 모두를 평가하는데 흔히쓰인다. 10개 도메인에 걸친 150개 범위 내 의도와 총 23.7K 질의 (범위 내 22.5K, OOS 1.2K)를 포함한다. - **Banking:** Banking77은 짧은 은행고객 서비스 질의로 이루어진 단일 도메인 의도 데이터셋으로, 77개 세분의도 (예: 카드 문제, PIN 재설정)로 라벨링된다. 13,083개 예제를 포함하며의도 분포가 눈에 띄게 불균형하다. - **DBpedia:** DBpedia 기반분류는 Wikipedia 파생 설명 (주로 초록)을 온톨로지 범주 (예: 책·영화·동물·장소)로 매핑한다. 70-클래스, level-2 설정을 사용하고 학습 10K·시험 1K 인스턴스를 부분표집한다.

레벨 수가 ICL에 미치는 효과. long-context에서의 질의응답과 다중키 검색을 평가하기 위해 다음 데이터셋을 사용한다. - **LongHealth:** 긴 가상 환자 사례 기록으로 구축된 long-context 임상 다지선다 QA 벤치마크. 20개 사례 문서 (각 약 5.1K–6.8K 단어)를 포함하며, 이 기록에서표집한 200개 질문으로 평가한다. - **QASPER:** 전문 (全文) NLP 논문에기반한 정보 탐색 QA 벤치마크. 약 1.6K 논문에 대한 약 5K 질문을 포함하며, 각 논문의 전문을 문맥으로 사용한다. - **MK-NIAH:** RULER의 다중 키 needle-in-a-haystack 과제. 여러 키-값 사실이 긴 문맥에 삽입되고 모델은질의된 키의 값을 검색해야 한다.

B.2 추가 결과: BABILong

Hope(Sleep)를 BABILong 벤치마크에서 (1) 대형 모델 (GPT-4, GPT-4o-mini), (2) RAG를 붙인 중규모 Llama-8B, (3) RMT·ARMT·Titans 등 최첨단 소형 long-context 모델과 비교해 평가한다. 모든 소형 모델은 공식 BABILong 학습 프로토콜로파인튜닝된다. 그림 6에서 보듯, 대형 모델은 문맥 길이가 늘수록 급격히저하되어 128K–256K 토큰을 넘으면 실패한다. RAG는 더 긴 문맥에서 강건성을개선하지만 시퀀스 길이가 늘수록 여전히 저하된다. 파인튜닝된 소형 모델 중 Titans·ARMT·Hope는 약 100만 (1M) 토큰까지비슷하게 작동하나, 그 이후 Titans와 ARMT는 급격히저하되는 반면 Hope는 1000만 (10M) 토큰의 극단적 길이에서도 안정적이다. 이 강건성은 단명하

는활성화를 압축된 파라미터 표상으로 변환하는 Sleep의 명시적공고화·자기 개선 메커니즘에서 비롯한다. 공고화 중 더 높은수준의 추상을 학습함으로써 Hope는 과제 관련 정보를 더 효율적으로 유지하여 문맥 길이 증가에 대한 민감도를 줄인다. 끝으로, Hope를 포함한 모든 소형 모델은 파인 튜닝 없이는 상당한 성능 하락을 겪는다. 초장문맥을 압축하려면 저빈도 기억 레벨에 충분한 용량이 필요하며, 파인 튜닝은 이구성요소들이 빠르게 적응하고 추론 중 고빈도 기억과 효과적으로 협응하게 한다.

B.3 추가 실험: ARC

few-shot ARC 실험 프로토콜을 따르되 우리 수면 패러다임에 맞게 조정한다. 백본으로 Llama-3.2-1B를 사용한다. 표준 구성에서 풀 수 없는 과제를 피하기 위해 데이터 일부를 필터링하여 학습용 11개 과제와 평가용 held-out 8개 과제를 얻는다. 각 Sleep 사이클 동안 모델은 먼저 이전 기억을 공고화한 뒤, few-shot 시연으로부터 합성 경험을 생성하여 꿈꾼다. 각 과제마다 60개의 꿈을 샘플링하고 그중 45개를 기각한다. 시험 시, 각 미지의 과제에 대해 모델은 5개의 꿈을 생성하고 held-out 출력을 예측하기 전에 이를 독립적으로 적용한다. 정답을 내는 꿈의 비율을 보고한다. 기준선으로 Zweiger et al. (2025)을 따라 (i) ICL, (ii) TTT + 합성 업데이트 (꿈꾸기 없음), (iii) SEAL을 사용한다. 이 설정에서 Sleep은 80%의 성공률을 달성하여 다른 방법보다 높다.

B.4 추가 세부 사항 (Additional Details)

표 5: GRPO, SFT, Sleep의 학습 구성.

파라미터	GRPO	SFT	Sleep
Learning Rate	5×10^{-6}	5×10^{-6}	5×10^{-6}
Effective Batch Size	32	32	32
Training Steps	500	100	100
LoRA Rank (r)	64	64	64
LoRA Alpha (α)	128	128	128

B.5 효율성 (Efficiency)

동일한 스텝 수로 보면 SFT가 우리 방법보다 4배 효율적이지만, 그 성능은 비교 대상이 되지 못한다. 이에 따라 우리는 특정 성능을 목표로 할 때의 효율도 비교한다. 모델들이 AIME-24, AIME-25, HMMT-25에서 동일한 성능을 달성하도록 학습했다. 이 경우 SFT는 우리 설계의 성능에 맞추기 위해 각각 4.3배, 3.6배, 4.8배의 wall-clock 시간을 필요로 한다. 따라서 이 관점에서, 특정 성능을 목표로 할 때 Sleep은 매우 효율적이다.

부록 그림 (Appendix Figures)

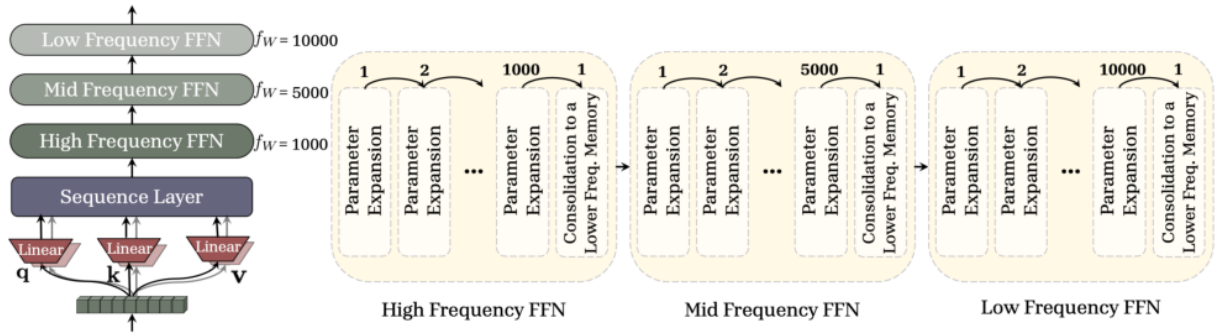


그림 7: 다중 빈도 기억 계층. 업데이트는 반복적 파라미터 확장을 통해 고빈도 FFN 으로 들어가며, 윈도우 f_W 가 만료되면 지식이 중빈도, 이어서 저빈도 FFN 으로 공고화된다 ($1k \rightarrow 5k \rightarrow 10k$). 출처: 원저자, arXiv:2606.03979 Fig.7

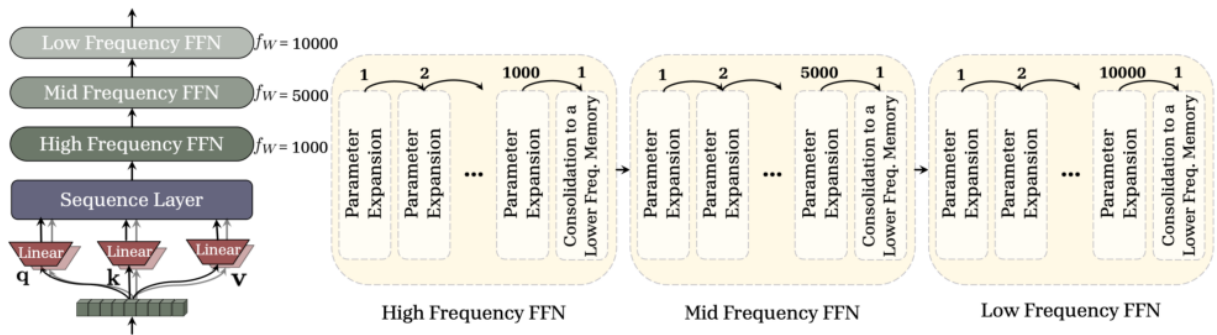


Figure 7: Multi-frequency memory hierarchy. Updates enter the High-Frequency FFN via repeated Parameter Expansion; when the window f_W expires, knowledge is Consolidated to the Mid- and then Low-Frequency FFNs ($1k \rightarrow 5k \rightarrow 10k$).

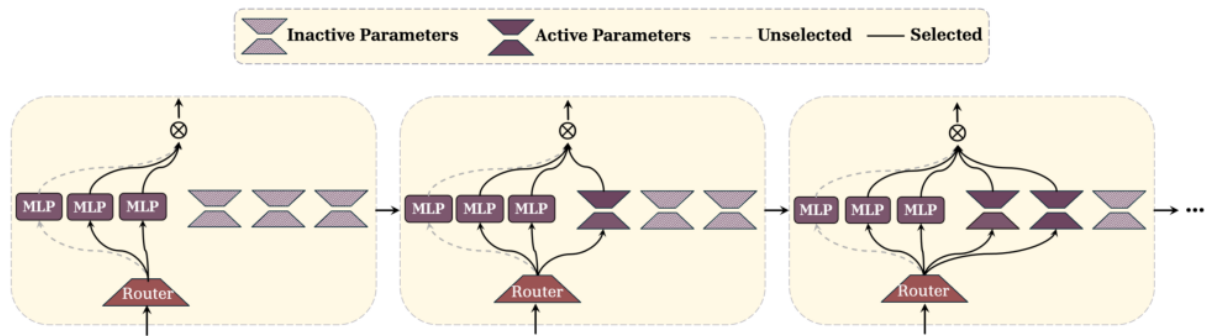


그림 8: 라우팅된 전문가 업데이트에 의한 기억 공고화. Sleep 사이클을 거치며 (좌→우) 라우터가 소수의 전문가 집합을 선택·업데이트 (실선) 하고 나머지는 비활성 (빛금) 으로 남겨, 간섭을 제한하면서 용량을 확장한다. 출처: 원저자, arXiv:2606.03979 Fig.8